

GPU-PIM Workload Bottleneck Analyzer

GPU, Memory Systems, PIM/NMP를 연결한 병목 분석 및 오프로딩 의사결정 연구

논문형 포트폴리오 기술 보고서

Roofline 분석, CUDA microbenchmark, Nsight Compute counter, SAIT PIMSimulator, size sweep을 하나의 재현 가능한 분석 파이프라인으로 통합했다.

9	v6	30	2.36x
CUDA workloads	cache/stall-aware model	automated tests	modeled policy speedup

Final project review: 2026-07-22

Target GPU: NVIDIA GeForce RTX 2080 Ti | CUDA 11.0 | Nsight Compute 2020.1.1

Executive Summary

이 프로젝트는 GPU kernel의 성능 문제를 단순히 memory-bound 또는 compute-bound로 분류하는 데서 끝나지 않고, 어떤 workload를 PIM/NMP로 옮길 가치가 있는지까지 근거 수준과 함께 제시한다. GPU runtime은 CUDA event로 측정하고, Roofline으로 1차 병목을 판별한 뒤, Nsight Compute의 DRAM, cache hit, scheduler, long-scoreboard 신호로 원인을 세분화한다. 마지막으로 analytical PIM cost와 SAIT PIMSimulator 결과를 결합해 offload policy의 전체 실행시간을 비교한다.

최종 결론. 9개 workload 중 vector_add, saxpy, random_gather, reduction, scan, matrix_transpose, gemv는 PIM/NMP 탐색 후보로 남았고, 높은 reuse의 matrix_mul_tiled와 compute-bound cublas_sgemm은 GPU 유지로 분류됐다. 현재 calibration set에서 v6는 7개 positive와 2개 negative control을 모두 구분했다. 단, 이는 독립 test accuracy가 아니라 설계에 사용한 labeled set에서의 정합도다.

핵심 결과

항목	결과	해석
Measured GPU-only runtime	10.963 ms	9개 workload CUDA event 합
Final v6 policy estimate	4.638 ms / 2.36x	4개 simulator-backed + 3개 analytical fallback
Simulator coverage	4/9 (44.4%)	vector_add, saxpy, reduction, gemv
Calibration controls	3/3 pass	GEMV positive, tiled/cuBLAS GEMM negative
Size stability	3 scales	small/medium/large에서 7 PIM : 2 GPU 유지

보고서 구성

1. 문제와 배경 · 2. 시스템 구조 · 3. 분석 방법 · 4. 구현 발전 과정 · 5. 실험 환경 · 6. Roofline 결과 · 7. NCU/cache 분석 · 8. PIM simulator 정렬 · 9. size sweep · 10. 검증과 한계 · 11. 재현 절차 · 12. 결론과 참고문헌

1. 문제 정의와 배경

1.1 GPU 병목을 왜 구분해야 하는가

GPU는 많은 thread를 동시에 실행해 memory latency를 숨기고 높은 연산 처리량을 낸다. 그러나 모든 kernel이 같은 이유로 느린 것은 아니다. arithmetic intensity가 낮으면 DRAM에서 데이터를 운반하는 속도가 한계가 되고, intensity가 높으면 SM의 연산 처리량이 한계가 된다. bandwidth도 compute도 충분히 쓰지 못한다면 irregular access, cache miss, dependency stall, divergence 또는 synchronization이 원인일 수 있다.

1.2 PIM/NMP가 해결하려는 문제

Processing-in-Memory(PIM)과 Near-Memory Processing(NMP)은 데이터를 연산기로 계속 가져오는 대신 memory 근처에서 일부 연산을 수행한다. 따라서 낮은 arithmetic intensity, 큰 memory traffic, 낮은 cache reuse, 높은 memory stall을 가진 workload가 후보가 된다. 반대로 dense GEMM처럼 cache/shared-memory reuse와 연산 밀도가 높은 workload는 GPU가 더 적합할 가능성이 크다.

GPU에 남길 신호

- 높은 arithmetic intensity
- 높은 L2/cache reuse
- compute-heavy operation
- 최적화 library 활용 가능

PIM/NMP를 검토할 신호

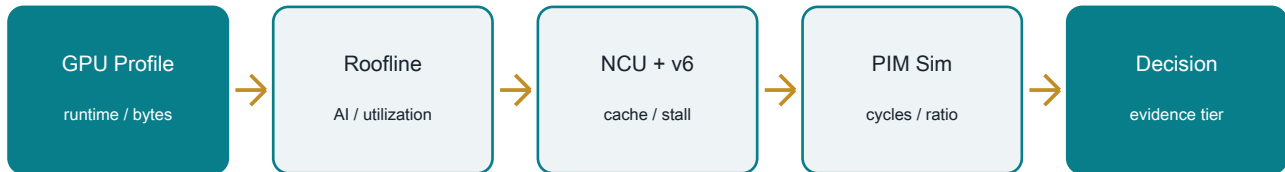
- 낮은 arithmetic intensity
- DRAM saturation 또는 memory stall
- irregular/streaming access
- 낮은 reuse와 높은 데이터 이동

1.3 프로젝트 질문

본 프로젝트의 질문은 세 가지다. (1) GPU kernel의 병목을 정량적으로 구분할 수 있는가? (2) 낮은 AI만 보는 naive rule보다 cache, stall, reuse, transfer risk를 포함한 모델이 더 합리적인가? (3) GPU 측정과 PIM simulator의 서로 다른 시간축을 섞지 않고 end-to-end 의사결정을 만들 수 있는가?

2. 시스템 구조

입력, feature extraction, model comparison, simulator cross-check, report generation을 독립 module로 분리했다. 이 구조 덕분에 CUDA가 없는 로컬에서는 기존 CSV로 분석 코드를 검증하고, GPU 서버에서는 측정값만 새로 생성할 수 있다.



측정값 · 시뮬레이터값 · 추정값을 분리해 추적

그림 1. 측정에서 최종 의사결정까지의 데이터 흐름

2.1 입력 계층

입력	역할	현재 근거
gpu_profile.csv	runtime, FLOPs, theoretical DRAM bytes	RTX 2080 Ti CUDA event
ncu_metrics.csv	DRAM/cache/scheduler/stall counters	Nsight Compute 2020.1.1
gpu_benchmark_metadata.csv	complexity, reuse, transfer risk labels	calibration metadata
sait_pim_simulation.csv	PIM on/off cycles and speedup	SAIT PIMSimulator logs

2.2 출력 계층

각 run은 Roofline plot, model comparison plot, end-to-end plot, Markdown report를 생성한다. Size sweep은 세 problem size의 원본 CSV, 개별 report, 통합 summary CSV/Markdown/plot을 남긴다. 모든 중간 근거를 저장해 최종 숫자가 어디서 왔는지 역추적할 수 있다.

3. 분석 방법

3.1 Roofline 1차 분류

Arithmetic intensity(AI)는 byte당 수행하는 FLOP 수다. RTX 2080 Ti 설정에서 peak compute는 13.45 TFLOP/s, peak memory bandwidth는 616 GB/s이며 ridge point는 21.8344 FLOP/Byte다.

$AI = FLOPs / DRAM\ bytes$

Attainable performance = $\min(\text{peak FLOP/s}, \text{peak bandwidth} \times AI)$

Roofline utilization = $\text{achieved performance} / \text{attainable performance}$

3.2 feature_cost_v6

v6는 v4의 reuse-aware 비용 모델과 v5의 기본 NCU 보정 위에 cache hit, load/store efficiency, warp/branch efficiency, long/short scoreboard, barrier stall, scheduler supply, register pressure를 선택적으로 추가한다. 누락 counter는 0이라는 실측값으로 간주하지 않으며, 사용 가능한 12개 선택 counter 비율을 coverage로 계산해 보정 강도를 제한한다.

신호군	PIM opportunity에 미치는 영향	GPU 유지 신호
Cache	낮은 L1/L2 hit	높은 cache hit/reuse
Memory	높은 DRAM pressure/stall	효율적 coalescing과 locality
Scheduler	낮은 eligible warp supply	latency hiding 가능
Control	낮은 divergence/barrier risk	복잡한 control/sync는 offload 위험
Metadata	낮은 complexity, 높은 partitionability	compute/transfer/reuse risk

3.3 공통 정책 평가

AI-only, traffic-only, heuristic, analytical, feature-cost model은 후보 선택만 다르게 한다. 전체 runtime을 비교할 때는 모두 같은 최신 v6 PIM 시간표를 사용한다. 이렇게 해야 model별 speedup 공식 차이가 아니라 offload 결정 자체의 품질을 비교할 수 있다.

4. 구현 발전 과정

2026년 5월 4일 기본 analyzer에서 시작해 CUDA suite, PIM simulator, NCU counter, size sweep까지 단계적으로 확장했다. 각 버전은 직전 모델에서 실제로 확인된 false positive 또는 근거 부족을 해결하는 방향으로 설계됐다.

단계	구현	해결한 문제
v1-v2	Roofline + heuristic score + PrIM proxy	기본 병목과 literature category 연결
v3	GPU/PIM memory, compute, transfer, sync cost	낮은 AI만으로 생기는 false positive 감소
v4	data reuse gate + cuBLAS SGEMM control	GEMM을 단순 memory 후보로 오판하지 않음
v5	Nsight Compute DRAM/SM/cache pressure	실제 counter로 proxy 보정
v6	cache hit, scheduler, stall, coverage	latency/locality 원인과 부분 counter 불확실성 반영
Final	simulator ratio normalization + evidence tier	서로 다른 clock domain의 잘못된 합산 제거

주요 engineering 결정

- CUDA event runtime과 theoretical FLOPs/bytes를 분리 저장했다.
- 외부 PIM simulator를 vendoring하지 않고 adapter CSV schema로 연결했다.
- 오래된 NCU 2020.1.1에서도 동작하도록 optional metric과 NA 처리를 지원했다.
- benchmark는 --n, --rows, --cols, --iterations를 받아 반복 실험이 가능하다.
- 30개 automated test로 parser, model, simulator, end-to-end 계약을 고정했다.

코드 구성

영역	핵심 파일
분석	src/roofline.py, classifier.py, features.py
모델	src/cost_model_v3.py ... cost_model_v6.py
통합	src/model_comparison.py, simulator.py, end_to_end.py
측정	benchmarks/*.cu, scripts/profile_*.sh
보고	src/plot.py, src/report.py, main.py

5. 실험 환경과 workload

GPU server	Profiling stack
• Ubuntu 20.04 • GeForce RTX 2080 Ti • Driver 535.230.02 • CUDA toolkit 11.0 • Host compiler g++-9	• CUDA event runtime • nvprof raw logs • Nsight Compute 2020.1.1 • SAIT PIMSimulator • Python/pandas/matplotlib

5.1 Benchmark suite

Workload	의도	최종 역할
vector_add / saxpy	streaming bandwidth	PIM 후보
random_gather	irregular latency	PIM 후보
reduction / scan	collective memory primitive	PIM 후보, sync risk 포함
matrix_transpose	layout transformation	PIM 후보
gemv	low-reuse matrix-vector	PIM positive control
matrix_mul_tiled	reuse-aware custom GEMM	GPU negative control
cublas_sgemm	optimized dense compute	GPU negative control

5.2 Size sweep

Vector problem은 1,048,576 / 4,194,304 / 16,777,216 elements, matrix problem은 512 / 1024 / 1536 크기로 측정했다. 단일 크기에서 우연히 나온 결론인지 확인하기 위해 runtime, bandwidth, bottleneck, offload decision, analytical speedup 추세를 비교했다.

5.3 데이터 해석 원칙

Measured는 GPU runtime과 NCU counter에만 사용한다. Simulated는 PIMSimulator cycle/speedup에 사용하며, Estimated는 feature-cost와 end-to-end policy에 사용한다. 이 세 용어를 구분해 실제 PIM hardware 성능을 측정하는 것처럼 보이지 않도록 했다.

6. Roofline 및 GPU 측정 결과

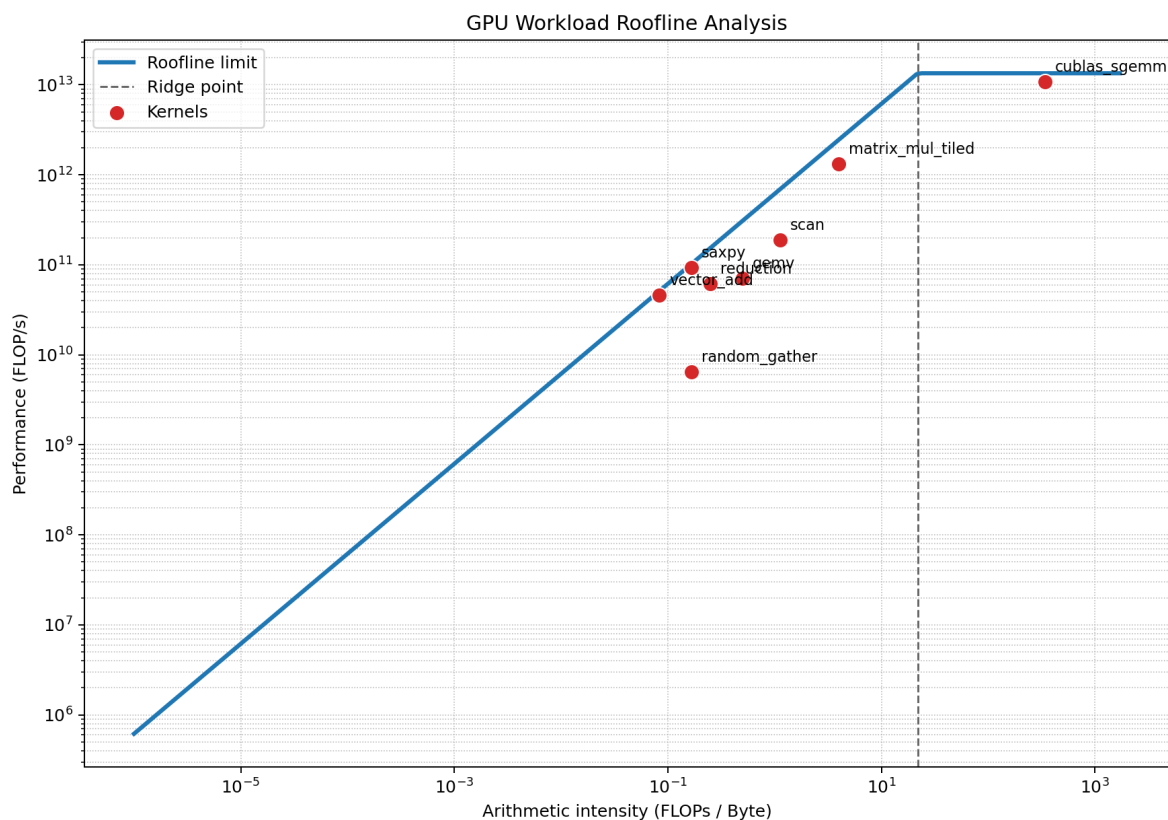


그림 2. RTX 2080 Ti에서 측정한 9개 workload의 Roofline 위치

그룹	대표 결과	판단
Streaming	vector_add/saxpy 약 555 GB/s, 90% Roofline	DRAM bandwidth 포화
Irregular	random_gather 38.8 GB/s, 6.3% Roofline	bandwidth보다 latency/locality 문제
Collective	reduction 40%, scan 27% Roofline	memory와 synchronization 혼합
Dense	cuBLAS SGEMM AI 341.3, 10.85 TFLOP/s	명확한 compute-bound control
Reuse	tiled GEMM AI 3.97, L2 hit 98.34%	낮은 AI만으로 PIM 판정하면 안 됨

Roofline만 보면 matrix_mul_tiled도 memory 영역에 있지만 높은 L2 reuse 때문에 PIM 후보가 아니다. 반대로 random_gather는 DRAM 대역폭을 포화하지 못해도 long-scoreboard stall이 매우 높으므로 near-memory execution의 latency 절감 가능성을 검토할 가치가 있다. 이 두 사례가 NCU-aware v6가 필요한 이유다.

7. Nsight Compute와 v6 결과

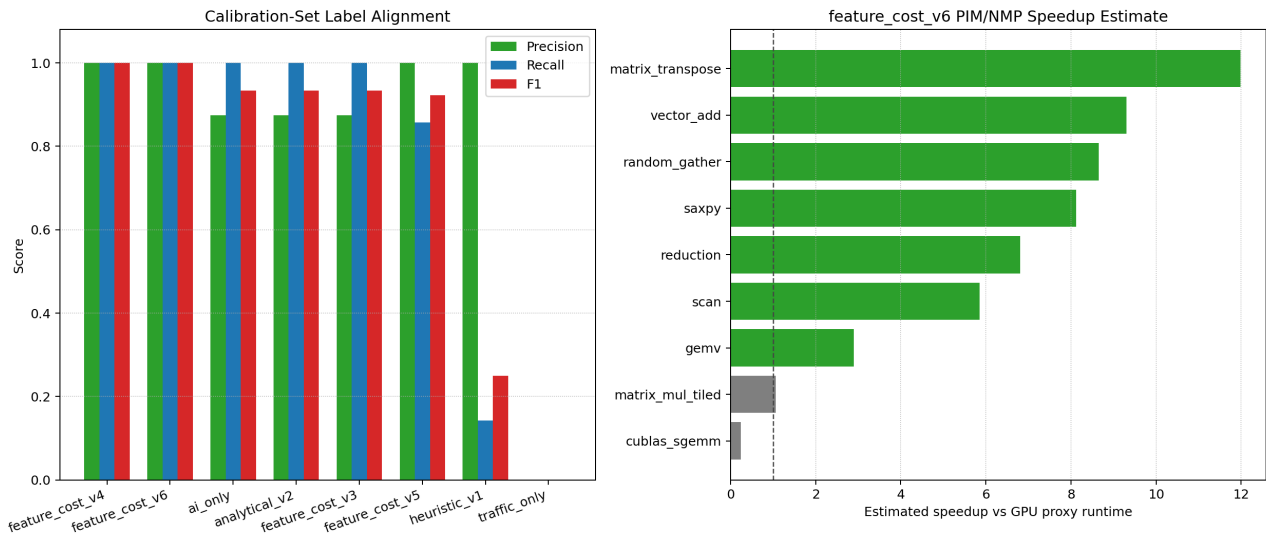


그림 3. Calibration-set label alignment와 최신 v6 analytical speedup

Kernel	핵심 NCU 신호	v6 해석
vector_add	SOL DRAM 85.41%, long scoreboard 91.20%	streaming PIM opportunity
random_gather	L2 hit 9.99%, long scoreboard 98.30%	irregular latency opportunity
gemv	L1 87.78%, long scoreboard 67.50%	locality가 있으나 simulator positive
matrix_mul_tiled	L2 hit 98.34%	GPU locality advantage
cublas_sgemm	L2 hit 87.47%, compute-bound	GPU 유지

7.1 최종 결정

PIM/NMP 탐색 후보	GPU 유지
vector_add, saxpy, random_gather	matrix_mul_tiled
reduction, scan, matrix_transpose, gemv	cublas_sgemm

현재 NCU 버전은 12개 optional v6 counter 중 kernel당 4개를 제공해 coverage는 33%다. 따라서 v6는 cache/stall 신호를 사용하되 v5 추정에서 멀어지는 보정량을 coverage만큼만 적용한다. 이는 누락값을 실제 0으로 처리해 지나치게 확신하는 문제를 막는다.

8. PIM Simulator 통합과 시간축 정렬

8.1 발견한 핵심 문제

초기 구현은 SAIT PIMSimulator cycle에 임시 1 ns/cycle을 곱한 값을 RTX 2080 Ti runtime과 직접 합산했다. 그러나 simulator baseline과 GPU는 같은 clock domain도, 같은 architecture도 아니다. 이 방식은 cycle 추적에는 유용하지만 cross-platform runtime 비교에는 적합하지 않다.

```
raw_sim_time_ms = PIM cycles × cycle_time_ns / 10^6
scaled_PIM_time_ms = measured_GPU_time_ms / simulator_speedup
```

최종 구현은 raw time과 cycle을 evidence로 보존하고, 전체 합산에는 simulator가 같은 내부 환경에서 측정한 PIM-on/PIM-off speedup 비율을 실제 GPU runtime에 적용한다. Simulator가 없는 kernel은 analytical v6 time으로 fallback하며 그 개수를 report에 표시한다.

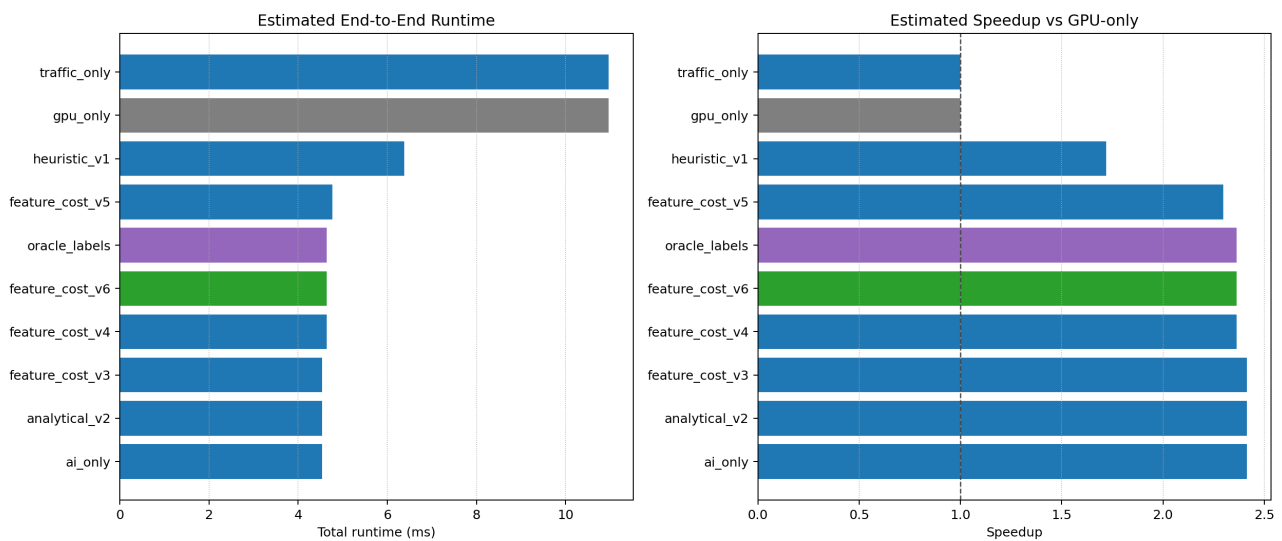


그림 4. GPU-only와 공통 PIM 비용표를 사용한 offload policy 비교

지표	최종 값	근거
GPU-only	10.963 ms	measured CUDA event 합
feature_cost_v6 policy	4.638 ms	4 simulator-scaled + 3 analytical
Modeled speedup	2.36x	실측 PIM speedup이 아님
Simulator coverage	44.4%	4/9 mapped workload

9. Problem Size Sweep

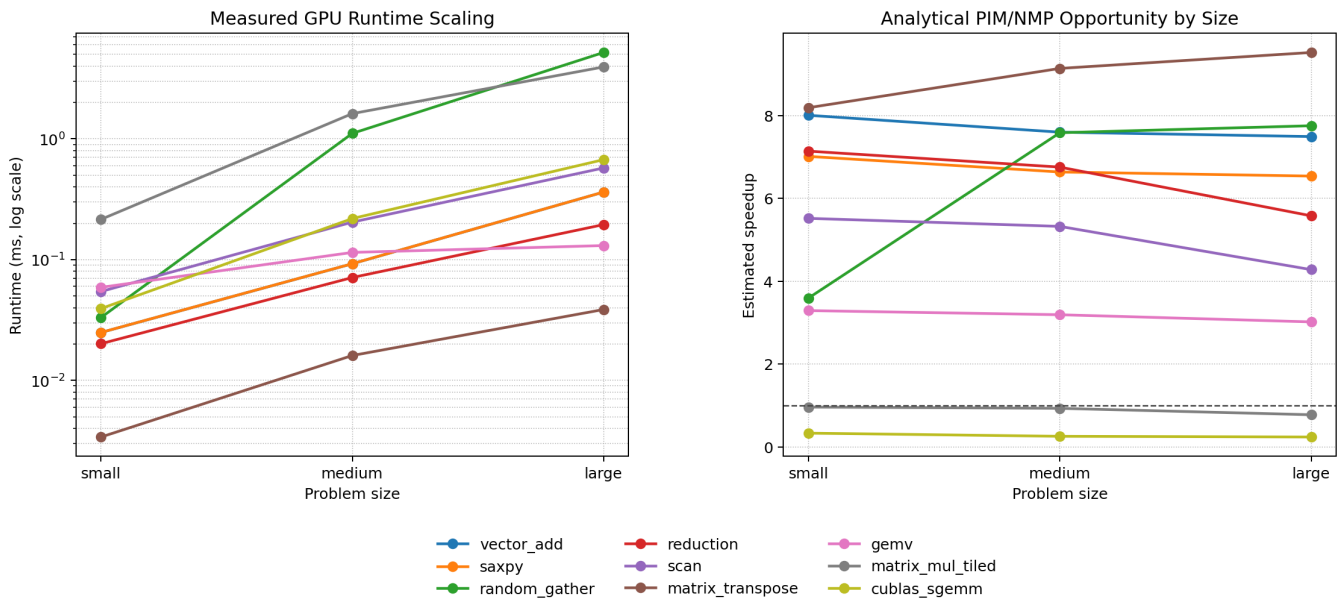


그림 5. 세 input size에서의 GPU runtime과 analytical PIM opportunity

Runtime은 입력 크기에 따라 정상적으로 증가하며 random_gather의 증가가 가장 가파르다. Streaming kernel은 큰 크기에서도 약 6.5-7.5배의 analytical opportunity를 유지한다. Matrix transpose는 높은 opportunity를 유지하고, tiled GEMM과 cuBLAS SGEMM은 1배 아래로 GPU 유지가 일관된다.

Workload	Small	Medium	Large
vector_add	PIM/NMP	PIM/NMP	PIM/NMP
random_gather	PIM/NMP	PIM/NMP	PIM/NMP
reduction	PIM/NMP	PIM/NMP	PIM/NMP
scan	PIM/NMP	PIM/NMP	PIM/NMP
gemv	PIM/NMP	PIM/NMP	PIM/NMP
matrix_mul_tiled	GPU	GPU	GPU
cublas_sgemm	GPU	GPU	GPU

세 scale에서 7 PIM/NMP : 2 GPU decision이 유지됐다는 것은 최소한 현재 synthetic suite 내에서는 단일 problem size의 우연에 의해 결정이 뒤집히지 않았음을 뜻한다. 하지만 size sweep은 independent workload validation을 대체하지 않는다.

10. 검증, 증거 수준, 한계

10.1 검증 범위

검증	결과	남는 위험
Automated tests	30 passed	실제 GPU/NCU 실행은 서버 필요
Calibration labels	F1 1.00 for v4/v6	held-out accuracy 아님
Control workloads	3/3 pass	control 수가 작음
Size sweep	3 scales stable	application diversity 제한
PIM simulator	4/9 mapped	kernel fidelity와 hardware 차이

10.2 Evidence tier

Tier	근거	현재 workload
A	Measured GPU + NCU + mapped PIM simulator	vector_add, saxpy, reduction, gemv
B	Measured GPU + NCU + analytical PIM	나머지 5개
C	Measured GPU + analytical PIM	NCU 없는 size sweep run

10.3 정직하게 남겨야 하는 한계

- 실제 PIM silicon에서 측정하지 않았다.
- 9개 synthetic/microbenchmark와 2개 negative control로는 일반화 성능을 주장할 수 없다.
- FLOPs/DRAM bytes는 benchmark 구조 기반 theoretical count다.
- NCU 2020.1.1은 최신 GPU/NCU보다 counter coverage가 낮다.
- Threshold와 analytical bandwidth/throughput 상수는 calibration 대상이다.
- End-to-end estimate는 data placement, offload orchestration, programming overhead를 완전히 모델링하지 않는다.

10.4 다음 연구 우선순위

가장 가치 있는 다음 단계는 model v7을 만드는 것이 아니라, threshold를 동결한 뒤 unseen application benchmark로 평가하는 것이다. 이어 다른 GPU architecture에서 NCU counter를 수집하고, UPMEM 또는 실제 PIM platform에서 최소한 vector/GEMV/collective workload를 검증해야 한다. 그 이후에 regression 또는 uncertainty calibration을 적용하는 것이 합리적이다.

11. 재현 절차

11.1 GPU 서버에서 실행

```
cd ~/GPU-Workload-Bottleneck-Analyzer
git pull
source .venv/bin/activate
pip install -r requirements.txt
python3 -m pytest
bash scripts/build_benchmarks.sh
bash scripts/profile_nvprof.sh
bash scripts/profile_ncu.sh
python3 scripts/parse_ncu_reports.py --input-dir profiles/ncu --output profiles/ncu_metrics.csv
python3 scripts/parse_sait_pim_logs.py --log-dir ~/pim-tools/pim-results \
--output simulators/sait_pim_simulation.csv --cycle-time-ns 1.0
python3 main.py --input profiles/gpu_profile.csv \
--paper-baseline paper_baselines/gpu_benchmark_metadata.csv \
--pim-simulation simulators/sait_pim_simulation.csv \
--ncu-metrics profiles/ncu_metrics.csv \
--output-dir outputs/gpu_profile_with_sait_pim_ncu \
--hardware-name "RTX 2080 Ti" \
--peak-flops 13450000000000 \
--peak-memory-bandwidth 616000000000
bash scripts/profile_size_sweep.sh
```

11.2 로컬 Mac으로 결과 가져오기

```
cd /Users/kingjung/Desktop/gpu-bottleneck-analyzer
git pull
bash scripts/fetch_server_results.sh gpu_profile_with_sait_pim_ncu
open outputs/gpu_profile_with_sait_pim_ncu/reports/analysis_report.md
open outputs/size_sweep/size_sweep_summary.md
open outputs/size_sweep/size_sweep.png
open output/pdf/gpu_pim_bottleneck_analyzer_portfolio_report.pdf
```

11.3 성공 판정

정상 완료 시 profiles/gpu_profile.csv에 9개 row, profiles/ncu_metrics.csv에 9개 kernel, simulator CSV에 4개 mapping, 최종 Markdown report와 3개 figure, size sweep 3개 profile과 summary/plot이 있어야 한다. `python3 -m pytest`는 30 tests passed를 출력해야 한다.

12. 결론

이 프로젝트는 GPU 병목 분석, memory-system counter 해석, PIM/NMP 후보 선별을 하나의 재현 가능한 도구로 연결했다. 특히 낮은 AI만 보는 단순 rule에서 시작해 reuse-aware model, NCU cache/stall-aware v6, simulator time-domain normalization으로 발전했다는 점이 핵심이다. 최종 결과는 7개 PIM/NMP 탐색 후보와 2개 GPU 유지 workload를 일관되게 구분하며, 어떤 결과가 measured, simulated, estimated인지 명시한다.

완성도의 기준은 더 높은 숫자가 아니라, 결과를 재현하고 한계를 설명할 수 있는가이다. 현재 저장소는 포트폴리오와 후속 연구에 사용할 수 있는 완결된 prototype이다. 다음 단계는 feature를 계속 추가하는 것이 아니라 held-out application과 실제 PIM platform에서 검증 범위를 넓히는 것이다.

References

- [1] S. Williams, A. Waterman, D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," CACM, 2009.
- [2] NVIDIA, Nsight Compute Kernel Profiling Guide: Roofline, Memory Workload Analysis, Scheduler Statistics.
- [3] J. Gómez-Luna et al., "Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture," IEEE Access, 2022.
- [4] Samsung Advanced Institute of Technology, SAITPublic/PIMSimulator: Processing-In-Memory Simulator.
- [5] NVIDIA, CUDA C++ Programming Guide and cuBLAS documentation.

Repository artifact: GPU-Workload-Bottleneck-Analyzer

Final implementation includes source, tests, raw/parsed profiles, simulator adapter, generated reports, and this portfolio document.